Sub-Project: Data-Driven Verification of Financial Markets

Financial Computing & Algo Trading Section

Spring 2026

Abstract

This sub-project aims to take the built high-performance Limit Order Book (LOB) simulator and produce a formally verification using OCaml and the **Rocq** (formerly Coq) theorem prover, or Cameleer, a pared down subset of Rocq. While recent theoretical work has formalized Continuous Double Auctions (CDA), a gap remains between abstract algorithms and real-world data reconstruction. By validating our engine against NASDAQ historical data (LOBSTER), we aim to create an "Automated Auditor" that serves as a rigorous foundation for microstructure research, algorithmic trading backtesting, and undergraduate theses.

1 Introduction & Motivation

Financial research relies heavily on backtesting simulations. However, simulators built in imperative languages (Python, C++) often suffer from "state drift"—subtle bugs where the simulation diverges from exchange reality due to mutable state errors.

We propose a **Purely Functional** approach. By treating market states as immutable snapshots, we can:

1. Guarantee **Exact Replay** for debugging.
2. Use **Formal Verification** to prove that the engine never violates market invariants (e.g., crossing the book).
3. Use the verified engine as an **Oracle** to validate the integrity of third-party data.

2 The Academic Landscape & The "Research Gap"

2.1 Prior Art

The application of formal methods to finance is a rapidly evolving field.

- **Call Auctions:** Sarswat and Singh (2020) [1] provided the foundational mechanized proofs for static batch auctions.
- **Continuous Double Auctions (CDA):** Very recently, Garg and Sarswat (2024) [2] extended this to verify continuous auctions with integer quantities, effectively solving the core *algorithmic* verification challenge using Red-Black trees.
- **Industrial Verification:** Firms like Imandra have applied these techniques to commercial venues (e.g., Goldman Sachs' Sigma X) [4].

2.2 Our Novelty: Data-Driven Verification

Since the *algorithm* for CDAs is now theoretically verified, our project pivots to **Implementation Verification** and **Data Consistency**.

- **The Gap:** Theoretical proofs assume abstract inputs. They do not account for the complexities of reconstructing a book from a lossy, real-world data feed like LOBSTER (NASDAQ).
- **Our Contribution:** We will build an engine that is not only mathematically correct but empirically validated. We verify the *Reconstruction Pipeline*:

  $$VerifiedEngine(LOBSTER_Msg_t) \equiv LOBSTER_Snapshot_t$$

  If our formally verified engine diverges from the provided snapshot, we have identified an anomaly in the data feed or the exchange protocol itself.

3 Project Architecture

3.1 The Pipeline

To respect LOBSTER licensing (no redistribution) and maximize HPC performance, we decouple data from logic:

1. **Preprocessing (Local):** A Python/Polars tool converts raw CSVs into a strict, compressed Binary format (handling Integer quantities to avoid floating-point drift).
2. **Simulation (HPC):** The OCaml engine streams binary data, rebuilding the LOB state using immutable trees (Red-Black or AVL).

3.2 Verification Methodology (Rocq)

We will use the **Rocq** Proof Assistant (formerly Coq) to prove the following properties on our OCaml implementation:

- **Safety:** *No Crossed Markets.* $(\forall b \in Bids, \forall a \in AsksPrice(b) < Price(a))$.
- **Liveness:** *Fairness.* Orders match strictly by Price-Time priority.
- **Liquidity Conservation:** The volume in the book plus volume traded must exactly equal the sum of all order inflows minus cancellations.

4 Target Use Cases

1. **Market Microstructure Class:** Visualizing the exact evolution of the LOB state for teaching liquidity dynamics.
2. **Algo Trading Class:** A bug-free backtesting environment for student strategies.
3. **Research Infrastructure:** Replacing ad-hoc Python scripts (e.g., previous work by MSc students) with a formal standard for future PhD work.

5 References

References

[1] S. Sarswat and A. K. Singh. *Formally Verified Trades in Financial Markets*. ICFEM 2020. Springer. https://arxiv.org/abs/2007.10805
(Foundational definitions for verified auctions. See also the repository: https://github.com/suneel-sarswat/auction)

[2] M. Garg and S. Sarswat. *Efficient and Verified Continuous Double Auctions*. arXiv preprint, 2024. https://arxiv.org/abs/2412.08624
(Solves the theoretical verification of Continuous Auctions using Red-Black Trees.)

[3] R. Natarajan, S. Sarswat, and A. K. Singh. *Verified Double Sided Auctions for Financial Markets*. In *12th International Conference on Interactive Theorem Proving (ITP 2021)*. Leibniz International Proceedings in Informatics (LIPIcs), Vol. 193. https://doi.org/10.4230/LIPIcs.ITP.2021.28
(Extends verification to handle Multiple/Integer Quantities.)

[4] G. O. Passmore and D. Ignatovich. *Formal Verification of Financial Algorithms*. CADE 26. Springer, 2017. https://link.springer.com/chapter/10.1007/978-3-319-63046-5_3
(Industrial application of OCaml verification at Imandra/Goldman Sachs.)

[5] Y. Minsky and S. Weeks. *Caml trading: experiences with functional programming on Wall Street*. Journal of Functional Programming, 18(4):553–564, 2008. https://www.cambridge.org/core/journals/journal-of-functional-programming/article/caml-trading-experiences-with-functional-programming-on-wall-street/D521F350F223C4D13A7822E5896E49A4
(The seminal paper on using OCaml for HFT at Jane Street.)